1. Recommended Project Tech Stack()

- **Runtime Framework:** Node.js + Express.js (TypeScript)
- **Database:** Firebase Firestore (NoSQL Document Store) or SupaBase (forPostgreSQL)
- **File Storage:** Firebase Cloud Storage(will be changed if Supabase is used)
- **Authentication:** Firebase Auth (JWT verified via Firebase Admin SDK)
- **Security & Constraints:** Express-Rate-Limit (App level) + Firebase Security Rules

Work Split (6 Phases)*(will need to be adjusted in case of SupaBase)*

Phase 1: Firebase Initialization & Configuration

- Initialize the Express project repository with TypeScript.
- Set up core folders (controllers, middleware, routes).
- Install firebase-admin SDK and initialize it using the Firebase Service Account private key.
- Provision the Firebase Project in the Google Cloud/Firebase Console.
- Create the Firestore Database and establish the challenges, teams, and submission_logs collections.
- Configure Firebase Cloud Storage buckets for event media assets.

Phase 2: Firebase Auth & Security Middleware

- Build the team authentication endpoint. When a team leader logs in with their team identifier/email, mint a custom token using admin.auth().createCustomToken().
- Implement an Express authentication middleware that intercepts requests, decodes the Firebase ID token (admin.auth().verifyIdToken()), and attaches the team context to the request.
- Integrate express-rate-limit on submission routes to ensure teams cannot exceed 5 guess submissions per minute.

- Configure basic Firestore Security Rules in the console to prevent any direct unauthorized modifications outside the admin SDK.

Phase 3: The Challenge & Verification Engine

- Build the GET /api/challenge/current endpoint.
- Have the endpoint fetch the logged-in team's document, pinpoint their active current_challenge_id, retrieve that challenge, and strip the answer_hash field out entirely before returning the data to the frontend to ensure no data leaks.
- Build the POST /api/challenge/submit endpoint.
- Sanitize inputs (convert text to lowercase, strip trailing/leading spaces).
- Validate the answer against the stored bcrypt hash. On success, use Firestore atomic operations (admin.firestore.FieldValue.increment) to update challenges_completed and cycle the team's current challenge ID pointer forward.

Phase 4: Administrative God-Mode Controls

- Construct the admin override endpoint (PUT /api/admin/teams/:id/advance) allowing event managers to manually increment a group's current_round or skip hard challenges completely without answering.
- Build a global control document endpoint (e.g., system/config) to toggle game states or lock entire rounds.
- Expose endpoints for the admin panel allowing managers to update answers or inject asset arrays (URLs pointing to Firebase Storage) dynamically without modifying backend code.

Phase 5: Live Analytics & Real-Time Leaderboard

- Create the leaderboard calculation service. Since Firestore does not support native SQL multi-column sorting natively, execute a clean multi-step sort in JavaScript: fetch all teams, sort descending by challenges_completed, and tie-break ascending by total_time_taken.
- Expose this data as an optimized JSON array for frontend queries.
- Create an admin endpoint (GET /api/admin/logs) to query and search through the submission_logs collection by team or timestamp.
- Write a simple controller utility to convert the sorted team standings directly into an exportable CSV buffer string.

Phase 6: Timer Interceptors, Edge-Case Audits & Deployment

- Implement backend timer logic validation. The submission controller must reject any incoming payloads if the server clock indicates the round countdown has officially hit zero.
- Perform end-to-end integration tests to guarantee that a team cannot read or deduce future challenges out of sequence.

- Deploy the Node.js backend to a hosting environment (like Render or Railway), map production environment tokens, and deliver clear API pathing lists to the frontend engineers.